



Need to create an dblink between 2 AWS Oracle RDS using the TCPS on port 2484
it raises an certificate error when testing the dblink with an select

Could you please share more details to help me troubleshoot the certificate error? Specifically:

1. Are both Oracle RDS instances using SSL with certificates issued by AWS?
2. Have you imported the root and intermediate certificates into the Oracle wallet?
3. What exact error message are you receiving when the SELECT is executed over the DB link?
4. Are you using mutual TLS (client and server certificates), or just server-side TLS?
5. Have you verified that the Oracle wallet is accessible and properly configured on the initiating RDS instance?

With this information, I can better assist you in resolving the issue.



- 1 yes
- 2 no, they are not an RDS Oracle Custom, so we don't have access to the SO
- 3 ORA-29024:Certificate Validation Failure
- 4 no, only server
- 5 no

Thanks for the clarification. I'll look into how to correctly configure a TCPS database link between two AWS Oracle RDS instances—where you don't have OS access—to avoid the ORA-29024 certificate validation failure. I'll also focus on how to manage SSL/TLS certificate validation under these constraints.



Create TCPS DB Link Between AWS Oracle RDS Instances Without OS Access

Establishing a Secure Oracle DBLink (TCPS) between AWS RDS Instances

Connecting two Oracle databases on Amazon RDS over TCPS (SSL/TLS on port 2484) requires enabling SSL on both instances and properly handling certificate trust. Below are the step-by-step instructions and best practices to set up a secure database link and resolve the **ORA-29024: Certificate Validation Failure** error.

Step 1: Enable SSL (TCPS) on Both RDS Oracle Instances

First, ensure that both the source and target RDS Oracle instances support SSL connections on port **2484** (the default TCPS port). In Amazon RDS, this is done by adding the **Oracle SSL option** to each

DB instance's option group:

- **Create or use a custom Option Group** for Oracle and add the **SSL** option. When adding, specify the SSL port (use 2484) and the appropriate VPC security group so the port is accessible [docs.aws.amazon.com](#)[docs.aws.amazon.com](#). For example, using the AWS CLI:

```
bash

aws rds add-option-to-option-group --option-group-name YourOptionGroup \
  --options 'OptionName=SSL,Port=2484,VpcSecurityGroupMemberships="sg-
<YourSGID>",OptionSettings=[{Name=SQLNET.SSL_VERSION,Value=1.2}]'
```

This attaches SSL to the instance's listener on port 2484[docs.aws.amazon.com](#)[docs.aws.amazon.com](#). (The `SQLNET.SSL_VERSION` setting can be used to enforce a TLS version, e.g., 1.2.)

- **Apply the option group** to your RDS instances. If the instances already exist, modify them to use the new option group (or associate it upon creation for new instances) [docs.aws.amazon.com](#)[docs.aws.amazon.com](#). Amazon RDS will then start a second listener on port 2484 for SSL, in addition to the regular listener on 1521/1522[docs.aws.amazon.com](#).
- **Verify the SSL listener:** Once the option group is active, the Oracle listener on the target should show a TCPS endpoint. You can confirm by examining the listener log or using the RDS console (CloudWatch logs) to see connections. An entry with `PROTOCOL=tcps` in the listener log indicates the SSL port is active[docs.aws.amazon.com](#).

Step 2: Configure Network Connectivity on Port 2484

Both databases must be able to communicate over the TCPS port:

- **Security Groups:** Ensure the security group of the target RDS allows inbound traffic on port 2484 from the source RDS instance (or its security group). Likewise, the source should allow outbound to 2484. The `VpcSecurityGroupMemberships` field in the SSL option should include the security group that permits this traffic[docs.aws.amazon.com](#).
- **Networking:** If the RDS instances are in different VPCs or AWS Regions, make sure networking (VPC peering or inter-region connectivity) is in place so the source can reach the target on 2484. (If they are in the same VPC, ensure the subnet routing and security are configured accordingly.)

With SSL enabled and network open, you can now attempt to create the database link. However, without addressing certificate trust, a simple `SELECT * FROM dual@dblink` over TCPS may fail with **ORA-29024**. This error means the client side (source DB) could not validate the server's certificate.

Step 3: Obtain and Prepare the AWS RDS CA Certificates

The **ORA-29024: Certificate validation failure** occurs because the Oracle source DB does not trust the SSL certificate presented by the target DB. In Amazon RDS, the database's SSL certificate is signed by an Amazon CA. The source needs the Amazon **root** (and any intermediate) CA in its trust store (Oracle wallet) to validate the certificated[docs.aws.amazon.com](#)[docs.aws.amazon.com](#):

- **Download the RDS CA Certificate:** AWS provides public certificate bundles containing the AWS RDS root CAs. Download the bundle for your region (or the global bundle) from AWS documentation docs.aws.amazon.com. The bundle includes certificates such as **Amazon RDS Root 2019 CA** (for RSA 2048-bit, etc.). According to AWS, your client's trust store only needs the root CA certificate for the RDS instance docs.aws.amazon.com (the server's cert chain will include any intermediate, which chains up to this root).
- **Extract the required certificate:** Open the PEM file and identify the relevant root certificate (e.g., the one named "Amazon RDS Root CA 2019" or similar for your RDS's CA). You may have one or multiple certificates in the bundle; typically, include the root that matches the certificate authority of your RDS instance. (You can find which CA your RDS is using via the AWS console's **Certificate Authority** info on the instance or `describe-certificates` CLI.)
- **Optional – Identify the expected DN:** Note the **Distinguished Name (DN)** of the server's certificate if possible. You can get this by running an Oracle client connection or using Oracle's `SSL_SERVER_CERT_DN` feature to enforce the expected certificate identity. For example, the DN might look like:
`CN=<instance endpoint>, OU=Amazon RDS, O=Amazon Web Services, L=Seattle, ST=Washington, C=US.`
 This will be used as a safety check in the DB link (next steps).

Step 4: Create an Oracle Wallet with the AWS Certificate

Oracle uses wallets to store trusted certificates. Since we do not have OS access on RDS to run `orapki` directly, we will create the wallet and import certificates *outside* and then upload it. The high-level process:

- **Create a wallet on a local machine** (or any host with Oracle client tools). Using the `orapki` utility (part of Oracle client), create a new wallet and add the downloaded Amazon RDS CA certificate as a trusted certificate. For example:

```
bash

orapki wallet create -wallet /tmp/rdswallet -pwd Welcome1 -auto_login
orapki wallet add -wallet /tmp/rdswallet -pwd Welcome1 -trusted_cert -cert rds-ca-cert.pem
```

This will produce a wallet with an auto-login keystore (`cwallet.sso`) and the actual PKCS#12 file (`ewallet.p12`). The wallet should now contain the Amazon RDS CA in its "Trusted Certificates" list (you can verify with `orapki wallet display`) docs.aws.amazon.com. Make sure **no user certificate or key** is needed in this wallet, since we are using one-way SSL (server authentication only).

- **Use an Auto-Login Wallet:** The `-auto_login` flag created an auto-login wallet (`cwallet.sso`) which can be used by Oracle without providing a password. This is important because on RDS we won't be able to interactively supply a wallet password. The presence of `cwallet.sso` allows Oracle to open the wallet automatically beny23.github.io.

Now we have the wallet files ready on our local machine. Next, we need to get these files into the RDS instance.

Step 5: Upload the Wallet to the RDS Instance (No OS Access)

Because RDS is a managed service with no direct shell access, we must use database utilities to upload the wallet files. AWS provides the **rdsadmin** packages and Oracle's UTL_FILE to help with this beny23.github.io.

- **Create an RDS directory for the wallet:** Use the Amazon RDS admin procedure to create a directory in the database's file system that we can write to. For example, to create a directory named **REMOTE_WALLET**:

```
sql

EXEC rdsadmin.rdsadmin_util.create_directory(p_directory_name => 'REMOTE_WALLET');
```

This creates a directory accessible via Oracle named **REMOTE_WALLET**. You can confirm its actual path with a query:

```
sql

SELECT directory_path FROM dba_directories WHERE directory_name = 'REMOTE_WALLET';
```

The path will be something like `/rdsdbdata/userdirs/01` or `/rdsdbdata/userdirs/02` beny23.github.io. Note this path for use in the DB link definition later.

- **Write the wallet files using UTL_FILE:** We will transfer `cwallet.sso` and `ewallet.p12` into that RDS directory. One method (as reported by an AWS RDS user) is to Base64-encode the wallet files and then decode them in a PL/SQL block on the RDS instance beny23.github.io. For each wallet file:
 1. On your local machine, Base64-encode the file (e.g., `base64 -w0 cwallet.sso > cwallet.b64`).
 2. Copy the base64 text output and use it in a PL/SQL script on the RDS to write the file. For example:

```
sql

DECLARE
  f UTL_FILE.file_type;
  base64_blob VARCHAR2(32767) := '<Base64 string here>';
BEGIN
  f := UTL_FILE.fopen('REMOTE_WALLET', 'cwallet.sso', 'wb', 32767);
  UTL_FILE.put_raw(f, UTL_ENCODE.base64_decode(UTL_RAW.cast_to_raw(base64_blob)));
  UTL_FILE.fclose(f);
END;
/
```

This will create `cwallet.sso` in the **REMOTE_WALLET** directory by decoding the base64 content beny23.github.io. Repeat for `ewallet.p12` (changing the filename accordingly).

- **Verify the files on RDS:** List the files to ensure they were created correctly:

```
sql

SELECT * FROM TABLE(rdsadmin.rds_file_util.listdir('REMOTE_WALLET'));
```

You should see `cwallet.sso` and `ewallet.p12` in the output, with file sizes matching your original wallet files beny23.github.io.

Now the RDS instance has an Oracle wallet containing the Amazon RDS CA certificate. The next step is to tell the database link to use this wallet.

Step 6: Create the Database Link Using TCPS and the Wallet

Create (or recreate) the DBLink on the source database with a connect string specifying TCPS and referencing the wallet directory:

- **TNS connect string with TCPS:** The DB link's USING clause (or TNS alias) should use `PROTOCOL=TCPS` and port 2484, pointing to the target RDS endpoint and service/SID. For example:

```
sql

CREATE DATABASE LINK TargetDB_SSL CONNECT TO target_user IDENTIFIED BY password
USING '(DESCRIPTION=
        (ADDRESS=(PROTOCOL=TCPS)(HOST=<target-endpoint>)(PORT=2484))
        (CONNECT_DATA=(SERVICE_NAME=<service-name>))
        (SECURITY=(MY_WALLET_DIRECTORY=<wallet_path>)
        (SSL_SERVER_CERT_DN="<Target_Cert_DN>")));
```

In the above:

- `MY_WALLET_DIRECTORY=<wallet_path>` should be the full path obtained earlier (e.g., `/rdsbdbdata/userdirs/02`) where the wallet files reside beny23.github.io. This informs Oracle which wallet to use for SSL trust.
- `SSL_SERVER_CERT_DN="<Target_Cert_DN>"` is optional but recommended. It specifies the expected Distinguished Name of the server's certificate. Oracle will then verify the server's certificate matches this DN beny23.github.io. This adds an extra check to prevent man-in-the-middle scenarios by ensuring the certificate's identity is exactly what you expect (for example, the CN matching the RDS endpoint's DNS name or a specific DN string provided by AWS).
- **No client certificate needed:** Because we are using server-side TLS only (no mutual authentication), you do **not** include any `SSL_CLIENT_AUTHENTICATION` parameter in the connect string – the wallet we provided contains only trusted certificates, not a client identity. The target RDS by default does not require client cert if the SSL option is used in one-way mode (Oracle's `SQLNET.SSL_CLIENT_AUTHENTICATION` is effectively `FALSE` on the server for one-way SSL stackoverflow.com). We only need the wallet on the client side to trust the server.

After creating the link, you can test it with a simple query (e.g. `SELECT * FROM dual@TargetDB_SSL;`). If everything is configured correctly, the query should succeed over the encrypted channel. The ORA-29024 error should be resolved now that the source trusts the certificate presented by the target.

Step 7: Verify Connection and Certificate Validation

With the DB link in place, verify that the connection is indeed using SSL and that no certificate errors occur:

- **Check for errors:** A successful **SELECT** over the link without ORA-29024 indicates the certificate was accepted. If you still get a certificate error, double-check that the wallet path is correct and accessible by the DB user, and that the wallet contains the correct CA certificate (e.g., the Amazon RDS root). Ensure the wallet files were not corrupted during base64 transfer (file sizes should match). Also verify that the connect string's **SSL_SERVER_CERT_DN** exactly matches the certificate's subject DN (if provided).
- **View encryption status** (optional): On the target database, you can query **V\$SESSION_CONNECT_INFO** to confirm the session uses TCPS and encryption. For example:

```
sql

SELECT sid, network_service_banner
FROM v$session_connect_info
WHERE sid = (SELECT sid FROM v$session
             WHERE audsid = USERENV('SESSIONID'));
```

The banner should indicate SSL/TLS encryption (e.g., showing cipher details).

- **Listener logs:** The target's listener log (accessible via CloudWatch if configured) will show an entry for the connection with **PROTOCOL=tcps** which confirms the SSL channel docs.aws.amazon.com.

If everything is correct, the DB link is now securely established over TLS.

Handling Certificate Validation and Bypass Options

It is **not recommended to bypass certificate validation**, as that undermines the security of the SSL connection. The proper solution is to install the trusted CA as we did above. Oracle 19c+ enforces strict certificate verification, whereas older versions might have been more lenient (which is why an 11g RDS instance might have worked without the wallet, but 19c started throwing ORA-29024) beny23.github.io.

Key points and best practices regarding certificate validation:

- **Include the correct CA in the wallet:** Ensure you have the full trust chain. Usually importing the root CA is sufficient docs.aws.amazon.com, but if AWS provided an intermediate certificate for your RDS, import that as well as a trusted cert. Oracle will fail to validate if any certificate in the chain is missing in the wallet or if the chain is untrusted beny23.github.io.
- **SSL_SERVER_CERT_DN usage:** We used **SSL_SERVER_CERT_DN** in the connect string to lock the DB link to the expected server certificate DN beny23.github.io. This does *not* bypass validation; rather, it adds an identity check. Both the DN and a valid trust chain must match for the connection to succeed. (If you omit this, Oracle will still connect as long as the certificate is trusted, but including it is a good practice for security.)
- **Disabling verification:** Oracle does not provide a straightforward parameter to "ignore certificate errors" in the TNS connection string or SQLNet for TCPS. There is no supported **SQLNET** setting to disable server certificate validation entirely, aside from installing a wallet that trusts the certificate. In other words, **to resolve ORA-29024 you must supply the proper certificates or use a certified trust store** docs.aws.amazon.com. Bypassing validation would defeat the purpose of SSL and is not advised.

- **Safe environment assumptions:** If for some reason you cannot manage a wallet, a theoretical workaround (not truly secure) is to use an **IP-secured network** and temporarily trust all certificates. In Java, for instance, one might skip certificate checks, but within the Oracle database context, there's no out-of-the-box way to do this. It's better to import the AWS CA as shown. Oracle's Advanced Security documentation confirms that ORA-29024 is expected if the client cannot validate the certificate presented pub.deadnet.se.
- **Keep the wallet updated:** AWS periodically rotates the RDS certificates (e.g., introducing new CA 2019, 2024, etc.). Monitor AWS announcements for certificate updates. If AWS issues a new CA, download the new cert and add it to the wallet (you can keep multiple trusted CAs in one wallet) [docs.aws.amazon.com](https://docs.aws.amazon.com/docs.aws.amazon.com). This way, you can avoid future ORA-29024 errors when RDS rotates the server certificates.

References and Further Reading

- AWS RDS User Guide – *Using SSL with an RDS Oracle DB Instance*: Explains enabling the SSL option and using a separate port for SSL [docs.aws.amazon.com](https://docs.aws.amazon.com/docs.aws.amazon.com).
- AWS RDS User Guide – *Downloading and using RDS SSL/TLS Certificates*: Instructs how to get the latest CA bundle and update client trust stores [docs.aws.amazon.com](https://docs.aws.amazon.com/docs.aws.amazon.com).
- AWS Knowledge Center – *Updating Trust Stores for Oracle SSL Connections*: Emphasizes that client applications (or DB links) must include the new CA in their trust store to connect via SSL docs.aws.amazon.com.
- *Tales about Software Engineering* Blog – **Fixing Oracle DB Link Certificate Errors in RDS**: A step-by-step example of resolving ORA-29024 on RDS by creating a wallet, using RDSADMIN to upload it, and specifying MY_WALLET_DIRECTORY in the DB link beny23.github.io. This illustrates the workaround for environments without OS access.

By following the above steps, you establish a secure, encrypted DBLink between the two RDS Oracle instances and eliminate the certificate validation errors. The key is properly trusting the AWS-issued certificates within the Oracle wallet on the source database, given the constraints of AWS RDS's managed environment. With this configuration, the SELECT over the DBLink should succeed securely (no ORA-29024), using TLS encryption on port 2484. beny23.github.io docs.aws.amazon.com



Fontes